

FUNDAMENTOS DE TEORÍA DE LA COMPUTACIÓN 2026
Trabajo Práctico Nro 3

Comentario: Tratar de hacer mínimamente los ejercicios del 1 al 5.

Ejercicio 1. ¿Qué es una MT universal?

Cito al libro:

“Una máquina de Turing universal es una máquina de Turing como cualquier otra, pero con la particularidad de que sus entradas representan máquinas de Turing, y su función de transición especifica cómo ejecutarlas (lo cual se ajusta más al funcionamiento de una computadora). En el caso más general, una máquina de Turing universal tiene como entradas pares de la forma $\langle M \rangle, w$, siendo la codificación de una MT M y w una entrada de M . Dada una MT universal U y una entrada $\langle M \rangle, w$, U ejecuta M a partir de w y responde como M (deteniéndose o no deteniéndose). Como toda MT, U tiene un conjunto de estados, un alfabeto y una función de transición.”

Su característica principal es que su función de transición le permite simular o ejecutar esa máquina externa a partir de la cadena recibida, respondiendo exactamente de la misma manera que lo haría la máquina original. Debido a su capacidad para cargar y ejecutar diferentes máquinas (algoritmos), este modelo se ajusta mucho más al funcionamiento de una computadora real o de propósito general

Ejercicio 2. Explicar cómo enumeraría los números naturales pares, los números enteros, los números racionales (o fraccionarios), y las cadenas de Σ^* siendo $\Sigma = \{0, 1\}$.

Números naturales pares:

En orden: 2, 4, 6, 8, 10, 12...

Números enteros:

Para no irme al $+\infty$ ni al $-\infty$, en zig-zag: 0, -1, 1, -2, 2, -3, 3, -4, 4, ...

Números racionales:

Lo tuve que buscar. Resulta que Georg Cantor en 1873 estuvo puteando con este temita hasta que lo resolvió...

El infinito de los reales es mayor que el infinito de los naturales, ergo los reales son NO enumerables. Pero los racionales SÍ se puede.

DEJO [VIDEITO](#).

Recomiendo mirar el videito, pero, básicamente:

Armás una tabla donde las filas son los numeradores y las columnas los denominadores. Recorrés la tabla en diagonal (flechitas que suben y bajan).

Si encontrás una fracción equivalente (como $1/2$ y $2/4$), la saltás para no repetir.

Cadenas de Σ^* siendo $\Sigma = \{0, 1\}$

En Orden Canónico, como hacíamos en OC.

Primero las más cortas. Si tienen el mismo largo, en orden numérico

0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, 0000, 0001, ...

Ejercicio 3. Dar la idea general de cómo sería una MT que, teniendo como cadena de entrada un número natural i , genera la i -ésima fórmula booleana satisfactible según el orden canónico. Comentario: asumir que existen una MT $M1$ que determina si una cadena es una fórmula booleana, y una MT $M2$ que determina si una fórmula booleana es satisfactible.

Idea general:

La máquina de Turing M va a generar una a una las cadenas del alfabeto lógico, siguiendo el orden canónico, y utilizará las máquinas $M1$ y $M2$ como subrutinas para validarlas y llevar un contador.

Dada una entrada i (un número natural), M hace:

1. Inicializa $n:=0$.
2. Genera la siguiente cadena v de un alfabeto que incluya todos los símbolos lógicos ($\vee, \wedge, \neg, \dots$) según el orden canónico.
3. Ejecuta $M1$ a partir de v para determinar si es una fórmula booleana. Si $M1$ rechaza, vuelve al paso 2.
4. Ejecuta $M2$ a partir de v para determinar si la fórmula es satisfactible. Si $M2$ rechaza, vuelve al paso 2.
5. Llegamos al paso 5 si y solo si ambas máquinas aceptaron. Entonces, hace $n:=n+1$
6. Si $n=i$, escribe v en su cinta de salida y acepta (q_A). Si no, vuelve al paso 2

Ejercicio 4. Sea $M1$ una MT que genera en su cinta de salida todas las cadenas de un lenguaje L . Dar la idea general de cómo sería una MT $M2$ que, usando $M1$, acepte una cadena w sii $w \in L$

Idea general:

Las máquinas de Turing generadoras escriben las cadenas del lenguaje iterativamente en una cinta de salida sin necesidad de recibir una entrada. Por lo tanto $M2$ simplemente dejará que $M1$ ejecute su generación y evaluará cada resultado.

Dada una cadena de entrada w , $M2$ hace lo siguiente:

1. Ejecuta la máquina generadora $M1$.
2. Cada vez que $M1$ escribe una nueva cadena v en su cinta de salida, $M2$ la compara con su entrada w .
3. Si $v=w$, entonces $M2$ se detiene y acepta (q_A).
4. Si $v \neq w$, $M2$ deja que $M1$ continúe su ejecución para generar la próxima cadena, y vuelve al paso 2.
5. Si en algún momento $M1$ termina su ejecución (es decir, ya generó todo el lenguaje) sin haber encontrado w , $M2$ rechaza (q_R). Si el lenguaje L es infinito (en RE), la ejecución de $M1$ no parará nunca. En este caso, $M2$ entrará en un bucle infinito (nunca se detendrá), reconociendo el lenguaje pero sin poder decidirlo

Ejercicio 5. El lenguaje $L_U = \{ \langle M \rangle, w \mid M \text{ acepta } w \}$ se conoce como lenguaje universal, y representa el problema general de aceptación. Probar que $L_U \in RE$. Ayuda: construir una MT que acepte L_U .

Para demostrar que el lenguaje universal L_U pertenece a RE, basta con construir una Máquina de Turing (MT) que lo reconozca.

Esta máquina es la Máquina de Turing Universal (MT U), ya que su función de transición le permite recibir como entrada y simular a cualquier otra máquina de Turing.

Idea general: Dada una cadena de entrada v , la MT U hace lo siguiente:

1. Valida la sintaxis de la entrada para asegurar que tenga la forma de un par $\langle M \rangle, w$, donde $\langle M \rangle$ es la codificación válida de una máquina de Turing y w es una cadena de entrada para esa máquina. Si no tiene este formato, U rechaza la entrada.
2. Simula la ejecución de M a partir de la cadena w .
 - En una cinta mantiene el estado actual de M (empezando por el estado inicial q_0).
 - En otra cinta mantiene el contenido de la cinta de M (la cadena w).
 - En otra cinta mantiene la tabla de transiciones de M (extraída de M).
 - Y con todo “en memoria” ya simula:
 - Busca en la tabla de transiciones qué debe hacer M dado su estado actual y el símbolo que está leyendo.
 - Actualiza el estado, escribe en la cinta simulada y mueve el cabezal según corresponda.
 - Y así va repitiendo (como una compu que ejecuta un programa, básicamente)
3. Entonces, MT U:
 - Si durante la simulación M alcanza su estado de aceptación (q_A), MT U se detiene y acepta.
 - Si M alcanza su estado de rechazo (q_R), MT U se detiene y rechaza.
 - Si M queda loopando, lógicamente MT U quedará loopando también. No hay problema con eso porque estamos en RE.

Ejercicio 6. Una función $f : A \rightarrow B$ es total computable si existe una MT M_f que la computa para todo elemento $a \in A$. Sea la función

$f_{01} : \Sigma^* \rightarrow \{0, 1\}$ tal que:

- $f_{01}(v) = 1$, si $v = \langle M \rangle, w$ y M para a partir de w .
- $f_{01}(v) = 0$, si $v = \langle M \rangle, w$ y M no para a partir de w o bien $v \neq \langle M \rangle, w$.

Probar que f_{01} no es total computable.

Ayuda: ¿con qué problema se relaciona dicha función?

Es similar al HP.

En HP, dado un caso de input particular, podemos chequear que pare o no, pero no podemos asegurarlo para todos los inputs.

Si $HP \in R$, entonces $R = RE$.

Es decir, si hubiera una MT que decide HP, también una MT que decide cualquier lenguaje de RE (Absurdo). La prueba es la siguiente:

- Para HP partimos del absurdo de que $HP \in R$. Sea M_{HP} una MT que decide HP.
- Y sea L algún lenguaje cualquiera de RE, Sea M_1 una MT que acepta L .
- Veamos que también existe una MT M_2 que decide L (y así L estará en R). Dado un input w , la MT M_2 hace:
 1. Ejecuta M_{HP} sobre el par $\langle M_1 \rangle, w$
 2. Si M_{HP} rechaza, significa que M_1 no para para w , y por lo tanto M_2 rechaza.
 3. Si M_{HP} acepta, significa que M_1 para a partir de w . Así, M_2 ejecuta M_1 sobre w y acepta si y sólo si M_1 acepta.

Claramente M_2 acepta y para siempre.

Esa prueba dice que si pudieras resolver el Halting Problem, podrías resolver CUALQUIER problema de la clase RE de forma perfecta. Como sabemos que hay problemas de RE que son imposibles de decidir, entonces el Halting Problem tiene que ser imposible también.

NO EXISTE esa máquina M_{HP} . Si vos corrés una máquina M_1 , y pasan 3 días y no terminó: ¿es porque es una cadena muy larga y le falta un segundo? ¿o es porque entró en un bucle infinito y no va a terminar nunca? No lo sabés. Por eso RE no es lo mismo que R.

SIEMPRE tenemos la incertidumbre de si. No habría diferencia entre "reconocer" (RE) y "decidir" (R).

Significado de total computable:

“Existe una MT que para siempre y te da el resultado. Esto equivale a decir que el lenguaje es R (Decidible)”

Entonces, el ejercicio 6, la idea es LA MISMA:

Partimos del absurdo de suponer que la f_{01} del enunciado existe.

Si esto fuera cierto, existiría una Máquina de Turing, llamémosla M_f , que es capaz de calcular esta función para cualquier entrada y siempre se detiene. Es decir, ante cualquier par $(\langle M_1 \rangle, w)$, M_f nos dice con un 1 si M para, o con un 0 si M no para, sin quedarse nunca loopeando.

Ahora, tomemos cualquier lenguaje L que sepamos **que es RE pero no es R** (por ejemplo, el lenguaje Universal L_U). Sea M_L la máquina que reconoce ese lenguaje L .

Si tuviéramos esa máquina M_f , podríamos construir una nueva máquina M decisora para decidir el lenguaje L de la siguiente manera.

Ante un input w :

- M decisora $\rightarrow$ ejecuta M_f sobre el par $(\langle M_L \rangle, w)$.
 - **Si M_f devuelve 0:** Significa que M_L no para para la entrada w . Por lo tanto, w no puede estar en el lenguaje. **M decisora rechaza.**
 - **Si M_f devuelve 1:** Significa que M_L sí para para la entrada w . Entonces, M decisora ejecuta la simulación de M_L sobre w re chill, total seguridad de que, eventualmente, se va a detener. Como M_f ya nos garantizó que la máquina va a terminar, sabemos que tarde o temprano M_L nos dará una respuesta (ya sea q_A o q_R)

En esta construcción, la máquina M decisora siempre se detiene.

Esto significaría que $L \in R$. Pero esto es un absurdo, ya que partimos de la base de que elegimos un lenguaje L que es RE pero no es R (como el lenguaje Universal o el propio HP).

Como la existencia de una MT que calcule f_{01} nos permitiría decidir lenguajes que ya demostramos que son indecidibles (implicaría de $R = RE$), la premisa inicial debe ser falsa.

Por lo tanto, f_{01} no es total computable.

Ejercicio 7. Responder breve y claramente cada uno de los siguientes incisos (en todos los casos, las MT mencionadas tienen una sola cinta):

7a. Probar que se puede decidir si una MT M , a partir de la cadena vacía λ , escribe alguna vez un símbolo no blanco. Ayuda: ¿Cuántos pasos puede hacer M antes de entrar en un loop?

Para probar que este problema es decidible (clase R), debemos demostrar que existe un número máximo de pasos N tal que, si la MT M no escribió un símbolo no blanco en esos N pasos, nunca lo hará. Quiero decir, TIENE ESTADOS FINITOS, si repite un mismo estado dos veces, va a entrar en bucle sí o sí.

Lo que condiciona a la máquina es que tiene finitos estados.

Si ponemos a correr la máquina M sobre la cinta de blancos, en cada paso puede pasar una de dos cosas:

- **Caso A:** Escribe un símbolo no blanco (0 o 1, por ej). En ese caso, detenemos el proceso y decimos SÍ.
- **Caso B:** Escribe un blanco (lo deja como estaba), cambia de estado y se mueve (izquierda o derecha).

Si la máquina hace $k + 1$ pasos y todavía no escribió un símbolo no blanco (siendo k la cantidad de estados de la máquina) significa que necesariamente repitió un estado! Es como eso que dice que si tenemos 366 personas, SÍ O SÍ, hay 2 que cumplen años el mismo día, ya que los días del año son 365, no queda otra. Acá es la misma lógica.

Ergo, podemos construir una MT D que haga:

- Cuenta cuantos estados k tiene la maquina M
- Simula M durante un máximo de $k+1$ pasos.
- Si en alguno de esos pasos M escribe algo distinto de blanco, MT D frena y dice qA
- Si pasan $k+1$ pasos y M solo escribió blancos, MT D frena y dice qR (sabemos que entró el loop de blancos)

7b. Probar que se puede decidir si una MT M que sólo se mueve a la derecha, a partir de una cadena w, para, Ayuda: ¿Cuántos pasos puede hacer M antes de entrar en un loop?

Similar al ejercicio 7a.

En este caso, lo que condiciona a la máquina es que **la cadena w es finita**.

Entonces, sí, se puede construir una MT D que decida si MT M va a frenar o no.

Idea general de MT D:

- Mide el largo n de la cadena w.
- Cuenta los estados k de la máquina M.
- Simula a M por un máximo de $n + k$ pasos.
- Resultado: Si M se detiene en alguno de esos pasos: MT D dice qA
- Si pasan los $n + k$ pasos y M sigue caminando a la derecha, MT D dice qR.

7c. Probar que se puede decidir si dada una MT M, existe una cadena w a partir de la cual M para en a lo sumo 10 pasos. Ayuda: ¿Hasta qué tamaño de cadenas hay que chequear?

En este caso, **lo que condiciona a la máquina es que tenemos un límite de 10 pasos**.

Cualquier cosa que haya en la celda número 11, la 100 o la un millón, es totalmente irrelevante. La máquina ni siquiera llega a ver esos símbolos antes de que se acabe su "crédito", no llega nunca.

Entonces, el conjunto de cadenas "interesantes" es finito:

- Son todas las cadenas posibles de longitud menor o igual a 10.
- Si el alfabeto es $\{0, 1, B\}$, la **cantidad de cadenas es finita y la cantidad de pasos también** (exactamente 3^{10}).

Podemos construir una MT Decisora que haga lo siguiente:

- Crea una lista de todas las cadenas posibles de longitud 10. LOS W SON FINITOS, ASÍ QUE PRUEBA TODAS.
- Para cada cadena w de esa lista:
 - Simula la MT M con la entrada w durante exactamente 10 pasos.
 - Si en algún momento de esos 10 pasos la máquina M se detiene (llega a qA o qR), la MT D frena y dice qA (encontró una).
 - Si termina de probar todas las cadenas de la lista y en ninguna paró antes de los 10 pasos, la MT Decisora dice NO.

7d. ¿Se puede decidir si dada una MT M , existe una cadena w de a lo sumo 10 símbolos a partir de la cual M para? Justificar la respuesta.

Parece igual al anterior, pero le cambiaron una sola palabra que te caga.

En el anterior (7c) el límite era de 10 pasos.

En este (7d), el límite es de 10 símbolos en la cadena, pero el tiempo (los pasos) es infinito.

La respuesta es: NO, no se puede decidir..

La diferencia:

- En el 7c (10 pasos): Tenías una cantidad finita de cadenas Y una cantidad finita de tiempo (pasos). Podías probar todo y terminar rápido.
- En este ejercicio (10 símbolos): Tenés una cantidad finita de cadenas (todas las w de largo 10), pero para cada una de esas cadenas, M podría no parar nunca.

El (7c) es como que le pone un timer, y cuando termina el timer, no le importa nada y da qR

En el (7d) no hay timer ¿cuanto esperamos? ¿3 meses? ¿6 meses? ¿10 años? ¿sigue ejecutando porque le falta un paso? ¿o es porque entró en un bucle infinito y no va a terminar nunca? No lo sabemos. HALTING PROBLEM.

NO ES DECIDIBLE, pero es reconocible.

$\notin R$

$\in RE$